

Desenvolvimento para Dispositivos Móveis

Chamada de API, Biblioteca
Frezzed, FutureBuilder

Universidade Federal do Rio Grande do Norte

Chamada de API

- Buscar dados da internet é **comum** para a maioria dos aplicativos.
- Dart e Flutter oferecem ferramentas, como o **pacote http**, para a realização dessa tarefa.

Chamada de API

- Os passos para essa implementação são:
 - Adicionar o pacote http.
 - Realizar uma requisição usando o pacote http.
 - Converter a resposta em um objeto Dart.
 - Exibir os dados com o Flutter.

Chamada de API

- Adicionar o pacote http

```
flutter pub add http
```

- Importar o pacote http

```
import 'package:http/http.dart' as http;
```

- Na plataforma Android também é necessário editar o arquivo AndroidManifest.xml para adicionar a permissão de Internet.

```
<uses-permission android:name="android.permission.INTERNET" />
```

Chamada de API

- O método `http.get()` retorna um **Future** que contém um Response HTTP.
- A classe `http.Response` contém os dados recebidos de uma chamada.

```
final res = await http.get(  
    Uri.parse(  
        'http://api.hgbrasil.com/finance/quotations?key=<suakey>',  
    ),  
);  
  
if (res.statusCode != HttpStatus.ok) {  
    throw 'Erro de conexão';  
}
```

Chamada de API

- Embora seja fácil fazer uma requisição de rede, trabalhar diretamente com um `Future<http.Response>` não é muito conveniente.
- Recomenda-se converter o objeto `http.Response` em um objeto Dart.

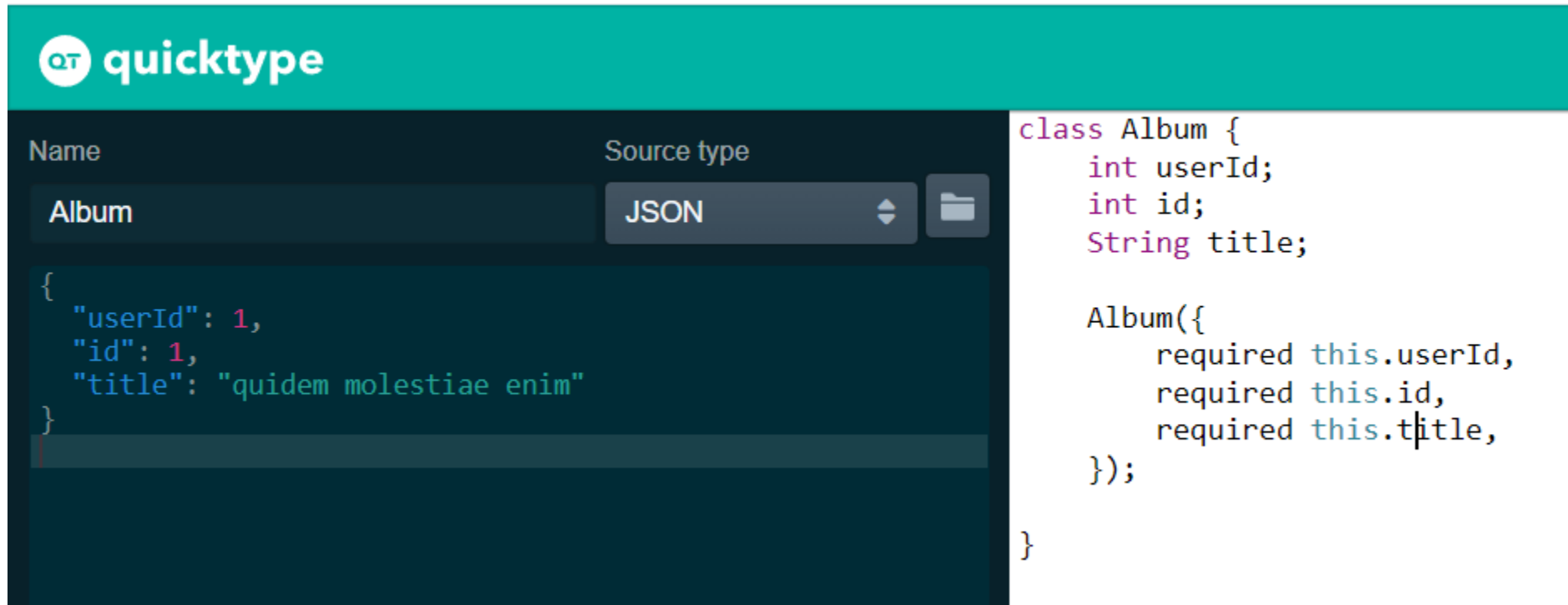
```
final res = await http.get(  
  Uri.parse(  
    'http://api.hgbrasil.com/finance/quotations?key=<suakey>',  
  ),  
);  
  
if (res.statusCode != HttpStatus.ok) {  
  throw 'Erro de conexão';  
}  
  
final data = jsonDecode(res.body);
```

Chamada de API

- É uma boa prática criar uma classe que contenha os dados da requisição de rede.
- Essa classe deve ter um construtor factory que cria um objeto a partir de JSON.
 - Converter JSON usando correspondência de padrões é uma das soluções para essa questão.
- Uma sugestão é utilizar o site <https://app.quicktype.io/> para descobrir o formato de um classe a partir de um objeto JSON.

Chamada de API

<https://jsonplaceholder.typicode.com/albums/1>



Chamada de API

- Implementar o construtor fromJson.

```
class Album {  
  final int userId;  
  final int id;  
  final String title;  
  const Album({  
    required this.userId,  
    required this.id,  
    required this.title,  
  });  
  factory Album.fromJson(Map<String, dynamic> json) {  
    return switch (json) {  
      {  
        'userId': int userId,  
        'id': int id,  
        'title': String title,  
      } =>  
        Album(  
          userId: userId,  
          id: id,  
          title: title,  
        ),  
      _ => throw const FormatException('Failed to load album.'),  
    };  
  }  
}
```

Chamada de API

```
Future<Album> fetchAlbum() async {  
    final response = await http  
        .get(Uri.parse('https://jsonplaceholder.typicode.com/albums/1'));  
  
    if (response.statusCode == 200) {  
        // If the server did return a 200 OK response,  
        // then parse the JSON.  
        return Album.fromJson(jsonDecode(response.body) as Map<String, dynamic>);  
    } else {  
        // If the server did not return a 200 OK response,  
        // then throw an exception.  
        throw Exception('Failed to load album');  
    }  
}
```

Opção para implementação de classes do modelo

- A biblioteca **frozen** do Dart é uma **ferramenta poderosa para simplificar a criação de classes imutáveis** e a manipulação de dados.
- Principais funcionalidades:
 - **Classes Imutáveis:** Facilita a criação de classes imutáveis, o que significa que uma vez que uma instância é criada, seus valores não podem ser alterados. Isso é útil para garantir que os objetos permaneçam consistentes e seguros para uso em ambientes concorrentes.
 - **Pattern Matching:** Suporta correspondência de padrões (pattern matching) que permite escrever código mais claro e conciso para lidar com diferentes casos de uso de uma classe. Por exemplo, ao lidar com diferentes estados em uma aplicação, você pode usar pattern matching para simplificar o código.

Opção para implementação de classes do modelo

- A biblioteca **freezed** do Dart é uma **ferramenta poderosa para simplificar a criação de classes imutáveis** e a manipulação de dados.
- Principais funcionalidades:
 - **Data Classes:** Cria classes de dados com propriedades definidas e fornece automaticamente implementações para métodos úteis, como `toString()`, `==` (igualdade), e `hashCode`.
 - **Factories e Cópia:** Oferece suporte para construtores de fábrica e métodos de cópia (`copyWith`). Isso facilita a criação de novas instâncias com modificações em relação às existentes sem alterar o original.
 - **Serialização e Desserialização:** Integra-se bem com bibliotecas de serialização como `json_serializable`, simplificando a conversão entre objetos Dart e JSON.

Biblioteca freezed

- <https://pub.dev/packages/freezed>

- Adicionar ao projeto

```
flutter pub add \  
  dev:build_runner \  
  freezed_annotation \  
  dev:freezed  
# if using freezed to generate fromJson/toJson, also add:  
flutter pub add json_annotation dev:json_serializable
```

- Gerar o código-fonte

```
dart run build_runner build
```

Biblioteca freezed

```
import 'package:freezed_annotation/freezed_annotation.dart';  
//make an object nicely readable in Flutter's devtool.  
import 'package:flutter/foundation.dart';  
  
// required: associates our `Album.dart` with the code generated by Freezed  
part 'album.freezed.dart';  
//Since our Album class is serializable, we must add this line  
part 'album.g.dart';  
  
@freezed  
abstract class Album with _$Album {  
  const factory Album({  
    required String userId,  
    required int id,  
    required String title,  
  }) = _Album;  
  
  factory Album.fromJson(Map<String, dynamic> json) => _$AlbumFromJson(json);  
}
```

Exibir os dados com o Flutter

- Para exibir os dados na tela, é usado o widget `FutureBuilder`.
- O widget `FutureBuilder` faz parte do Flutter e facilita o trabalho com fontes de dados assíncronas.
- Você deve fornecer dois parâmetros:
 1. O `Future` com o qual você deseja trabalhar. Neste caso, o future retornado pela função `fetchAlbum()`.
 2. Uma função builder que diz ao Flutter o que renderizar, dependendo do estado do `Future`: carregando, sucesso ou erro.

Exibir os dados com o Flutter

```
FutureBuilder<Album>(
  future: futureAlbum,
  builder: (context, snapshot) {
    if (snapshot.hasData) {
      return Text(snapshot.data!.title);
    } else if (snapshot.hasError) {
      return Text('${snapshot.error}');
    }

    // By default, show a loading spinner.
    return const CircularProgressIndicator();
  },
)
```

Exibir os dados com o Flutter

```
body: FutureBuilder(  
  future: dadosCotacoes,  
  builder: (context, snapshot) {  
    if (snapshot.connectionState == ConnectionState.waiting) {  
      return const Center(  
        child: CircularProgressIndicator.adaptive(),  
      );  
    }  
    if (snapshot.hasError) {  
      return Center(  
        child: Text(snapshot.error.toString()),  
      );  
    }  
    final data = snapshot.data!;  
  
    return Text("Dados lidos");  
  }  
);
```

Exibir os dados com o Flutter

- Observe que `snapshot.hasData` só retorna verdadeiro quando o snapshot contém um valor de dados não nulo.
- Como `fetchAlbum` só pode retornar valores não nulos, a função deve lançar uma exceção mesmo em caso de uma resposta "404 Not Found" do servidor.
- Lançar uma exceção define `snapshot.hasError` como verdadeiro, o que pode ser usado para exibir uma mensagem de erro.

Atividade

<https://console.hgbrasil.com/>



Referências

- <https://dart.dev/>
- <https://dart-tutorial.com/>
- <https://docs.flutter.dev/ui>
- <https://docs.flutter.dev/ui/widgets>
- <https://docs.flutter.dev/ui/layout>
- <https://docs.flutter.dev/resources/architectural-overview>
- <https://flutterguide.com/lifecycle-methods-of-flutter-widgets/>